

AWS/GCP 실전 마스터 1일차 교안

1교시: Docker 개념과 Compose 실습

개념 설명

Docker는 컨테이너 기반의 가상화 플랫폼으로, 애플리케이션을 실행하는 데 필요한 코드와 라이브러리, 설정 등을 하나의 경량 패키지(컨테이너)로 묶습니다 ¹. 컨테이너는 호스트 OS의 커널을 공유하여 VM보다 훨씬 적은 리소스로 동작하며, 애플리케이션 간 완전한 격리된 실행 환경을 제공합니다 ² ³. 예를 들어 Docker 공식 문서에 따르면 “컨테이너는 애플리케이션과 종속성을 빠르고 신뢰성 있게 실행환경 간에 이동시켜주는 표준 단위 소프트웨어”입니다 ¹. Docker는 이러한 컨테이너의 생성, 배포, 관리 과정을 자동화하고, Docker Hub 같은 이미지 저장소를 통해 컨테이너 이미지를 공유할 수 있게 합니다 ⁴.

도커 환경에서는 보통 **Docker 데몬(Docker Daemon)**, **Docker 클라이언트(Docker CLI)**, **Docker 레지스트리** 등의 구성요소가 있습니다. Docker 클라이언트는 사용자가 명령어를 입력하면 Docker 데몬에 요청을 전달하고, 데몬은 컨테이너 실행, 이미지 관리 등을 수행합니다. 또한 Docker 레지스트리(예: Docker Hub, AWS ECR, GCP GCR)를 통해 컨테이너 이미지를 저장·배포합니다 ⁴.

![[아키텍처 그림 삽입 위치]]

(위: Docker 아키텍처 예시 - Docker 클라이언트, 데몬, 이미지 레지스트리 구성)

Docker Compose는 다중 컨테이너 애플리케이션을 간편하게 관리해 주는 도구입니다. 여러 서비스(컨테이너)로 구성된 애플리케이션을 하나의 YAML 파일(`docker-compose.yml`)에 정의하고, 단일 명령으로 모든 컨테이너를 동시에 시작/중지할 수 있습니다. Compose를 사용하면 `docker build`와 `docker run`을 일일이 실행할 필요 없이, 구성 파일에 서비스별 이미지와 설정(포트, 환경변수, 볼륨 등)을 선언하여 손쉽게 일괄 처리할 수 있습니다 ⁵. 예를 들어 “Compose 파일을 사용하면 단 하나의 명령으로 전체 다중 컨테이너 애플리케이션을 시작하거나 중단할 수 있다”는 설명처럼, 복잡한 설정을 간단한 파일로 관리할 수 있습니다 ⁵.

실습 목표

- **Docker 컨테이너 이해:** Docker 이미지와 컨테이너의 개념을 익히고, 간단한 컨테이너를 실행해 봅니다.
- **Docker Compose 이해:** Docker Compose로 다중 컨테이너 서비스를 정의하고, `docker-compose up/down` 명령으로 애플리케이션을 기동/종료해 봅니다.

실습 절차

1. **Docker 설치 확인:** Docker Desktop(Windows/Mac) 또는 Docker Engine(리눅스)이 설치되어 있는지 확인하고, `docker --version`으로 정상 동작 여부를 확인합니다.
2. **간단한 컨테이너 실행:**
3. 예시: `docker run hello-world` 명령어로 Docker 설치를 검증하고, 간단한 “Hello World” 메시지가 출력되는지 확인합니다.
4. 예시: `docker run -d -p 80:80 nginx` 처럼 Nginx 이미지를 실행하여 웹 서버 컨테이너를 구동해 봅니다.
5. **Dockerfile 작성 및 이미지 빌드:**
6. 간단한 웹 애플리케이션(예: Python Flask 또는 Node.js Express)의 Dockerfile을 작성합니다. 예)

```
# 간단한 Node.js 앱 Dockerfile 예시
FROM node:14
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 3000
CMD ["npm", "start"]
```

7. `docker build -t myapp:latest .` 명령으로 이미지를 빌드하고, `docker run -d -p 3000:3000 myapp:latest` 로 컨테이너를 실행해 정상 동작하는지 확인합니다.
8. **Docker Compose 설정:**
9. `docker-compose.yml` 파일을 생성하여, 복수의 서비스를 정의합니다. 예) 웹 애플리케이션 컨테이너와 데이터베이스 컨테이너 두 개를 정의:

```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "3000:3000"
  db:
    image: mongo
    environment:
      - MONGO_INITDB_ROOT_USERNAME=admin
      - MONGO_INITDB_ROOT_PASSWORD=secret
```

10. 상기 파일을 준비한 후, `docker-compose up -d` 명령으로 두 개의 컨테이너를 동시에 시작합니다.
11. **컨테이너 실행 확인:**
12. `docker-compose ps` 명령으로 두 서비스(예: web, db)가 실행 중인지 확인하고, `docker-compose logs` 로 로그를 확인합니다.
13. 웹 애플리케이션 컨테이너가 제대로 실행되었다면 브라우저에서 `http://localhost:3000` 으로 접속해 결과 페이지를 확인합니다.
14. **Compose 종료:**
15. `docker-compose down` 명령으로 모든 컨테이너와 네트워크, 볼륨을 정리합니다.

실습 코드 예시

아래는 위 절차에서 사용한 예시 파일 및 명령어입니다.

• Dockerfile 예시 (Node.js 앱):

```
FROM node:14
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 3000
CMD ["npm", "start"]
```

• **docker-compose.yml** 예시:

```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "3000:3000"
    volumes:
      - ./app
  db:
    image: mongo
    environment:
      - MONGO_INITDB_ROOT_USERNAME=admin
      - MONGO_INITDB_ROOT_PASSWORD=secret
    ports:
      - "27017:27017"
```

• **실습 명령어 예시 (터미널):**

```
# Dockerfile 이미지 빌드 및 실행
docker build -t myapp:latest .
docker run -d -p 3000:3000 myapp:latest

# Docker Compose로 서비스 실행
docker-compose up -d
docker-compose ps
docker-compose logs web
docker-compose down
```

예상 결과

- **Docker 컨테이너 실행:** `docker run hello-world` 시 “Hello from Docker!” 메시지가 출력되고, Nginx 컨테이너의 경우 `docker ps` 에 Nginx가 실행중임이 표시됩니다.
- **Dockerfile 빌드/실행:** 앱 이미지 빌드 후 컨테이너가 정상 기동되며, `http://localhost:3000` 에서 애플리케이션 환영 메시지 등이 나타납니다.
- **Docker Compose:** `docker-compose up -d` 실행 시 `web_1` 과 `db_1` 두 컨테이너가 생성되고 실행됩니다. `docker-compose ps` 명령 출력에서 두 서비스가 “Up” 상태로 표시되며, `docker-compose logs` 명령으로 웹과 DB 로그가 출력됩니다.

요약 박스

- **혼자 해보기:**
- 새로운 컨테이너 서비스를 추가해 봅니다. 예: Redis 컨테이너를 `docker-compose.yml` 에 추가하고, `web` 컨테이너에서 연결 테스트를 수행해 보세요.
- Compose 파일에 설정 항목(환경변수, 볼륨)을 변경해 보세요. 예: `environment` 나 `volumes` 를 추가하여 설정이 적용되는지 확인합니다.

• 퀴즈:

- 컨테이너(Container)와 가상머신(VM)의 차이점은 무엇인가요?
- Docker Compose를 사용하면 어떤 장점이 있나요?
- `docker-compose up` 과 `docker-compose down` 명령은 각각 어떤 역할을 하나요?

• 체크리스트:

- [] Docker/Docker Compose가 설치되어 있는지 확인했나요?
- [] Dockerfile을 작성하고 이미지를 성공적으로 빌드했나요?
- [] `docker-compose.yml` 파일을 작성하고 서비스가 정상 실행되었나요?
- [] `docker-compose down`으로 리소스를 정리했나요?

2교시: GitHub Actions로 CI/CD 구성

개념 설명

CI/CD는 **지속적 통합(Continuous Integration)** 및 **지속적 배포/전달(Continuous Deployment/Delivery)**를 뜻합니다. CI는 코드 변경을 공유 저장소에 자주 머지(통합)하고 자동으로 빌드·테스트하는 과정을 말하며, CD는 이 과정을 이어 배포까지 자동화하여 소프트웨어 제공 주기를 단축합니다 ⁶. GitHub Actions는 GitHub 저장소 내에서 이벤트(예: 코드 푸시, PR 생성 등)를 트리거로 하여 자동으로 워크플로우를 실행하는 CI/CD 플랫폼입니다. Workflow는 `.github/workflows` 디렉터리에 저장되는 YAML 형식의 구성 파일로 정의하며, 하나 이상의 **job**으로 이루어집니다 ⁷ ⁸. 각 job은 일련의 **steps**로 구성되며, GitHub-hosted runner(또는 self-hosted runner) 위에서 실행됩니다 ⁸. 예를 들어, 워크플로우를 “push” 이벤트에 연결하면, 메인 브랜치에 커밋이 발생할 때마다 지정된 build/test/deploy 작업들이 순서대로 자동 실행됩니다 ⁹ ⁸.

!아키텍처 그림 삽입 위치

(위: GitHub Actions CI/CD 파이프라인 예시 - Push 이벤트로 워크플로우 트리거 및 여러 Job 수행)

GitHub Actions의 주요 구성요소는 다음과 같습니다.

- **이벤트(Events):** Push, Pull Request, Release 생성 등 워크플로우를 실행시키는 동작입니다.
- **워크플로우(Workflows):** YAML 파일로 정의된 자동화 프로세스로, 이벤트 발생 시 실행됩니다 ⁷.
- **잡(Jobs):** 워크플로우 내부에서 병렬 또는 순차 실행되는 작업 단위입니다. 각 잡은 단일 실행 환경(runner)에서 실행됩니다 ⁸.
- **스텝(Steps)과 액션(Actions):** 잡은 여러 스텝으로 구성되며, 각 스텝에서 스크립트를 실행하거나 `actions/checkout`, `aws-actions/configure-aws-credentials` 같은 **액션(Action)**을 호출할 수 있습니다.

실습 목표

- GitHub Actions를 사용해 코드를 자동 빌드(build)하고 간단한 테스트를 수행해 봅니다.
- 푸시 이벤트로 워크플로우가 실행되며, 성공 여부를 GitHub Actions 탭에서 확인합니다.

실습 절차

1. **GitHub 저장소 준비:** GitHub에 새로운 리포지토리를 생성합니다. (예: `actions-demo`) 프로젝트 코드는 간단한 `README.md` 와 Hello World 코드를 포함합니다.
2. **워크플로우 파일 작성:** 로컬에서 `.github/workflows/ci.yml` 파일을 생성하고 다음과 같이 작성합니다. (예시)

```
name: CI
on:
  push:
    branches: [ main ]
```

```

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '14'
      - name: Install Dependencies
        run: npm install
      - name: Run Tests
        run: npm test

```

위 예시에서는 main 브랜치에 푸시될 때 워크플로우가 시작되며, Node.js 환경 설정, 의존성 설치, 테스트 실행을 순차적으로 수행합니다.

3. **코드 커밋 및 푸시:** 작성한 워크플로우 파일을 커밋하고 `git push` 를 실행하여 GitHub에 반영합니다.
4. **Actions 탭 확인:** GitHub 저장소의 **Actions** 탭에서 해당 워크플로우가 트리거되어 실행됨을 확인합니다. 각 스텝 이름과 실행 결과(성공 또는 실패)가 표시됩니다.
5. **로그 확인:** 각 단계별 로그를 클릭하여 빌드 및 테스트 결과를 검토합니다. 테스트가 통과했다면 “Actions” 페이지에서 초록색 체크 표시로 나타납니다.

실습 코드 예시

- **GitHub Actions 워크플로우 예시** (`.github/workflows/ci.yml`):

```

name: CI
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.x'
      - name: Install requirements
        run: pip install -r requirements.txt
      - name: Run tests
        run: pytest

```

(Python 환경 예시)

- **예상 명령 실행 로그:** Actions 탭에서 각 스텝이 순차적으로 실행되고, 마지막에 모든 테스트가 성공적으로 끝나면 초록색 완료 표시가 나타납니다.

예상 결과

- **워크플로우 실행:** `push` 이벤트 발생 시 GitHub Actions에서 자동으로 워크플로우가 시작되며, 빌드 및 테스트 단계가 성공적으로 완료됩니다.
- **로그 확인:** 각 스텝 옆에 초록색 체크(성공)가 표시되고, 콘솔 로그를 통해 `npm install`, `pytest` 등의 출력 결과를 확인할 수 있습니다.

요약 박스

- **혼자 해보기:**
 - 워크플로우를 수정하여 **Pull Request** 이벤트에도 빌드가 실행되도록 해 보세요.
 - Lint나 코드 스타일 검사를 추가로 수행하도록 새로운 스텝(예: `eslint` 등)을 추가해 보세요.
- **퀴즈:**
 - GitHub Actions 워크플로우는 어디에 저장해야 하나요?
 - `runs-on` 옵션은 무슨 역할을 하나요?
 - 워크플로우를 트리거할 수 있는 이벤트 종류 2가지를 말해보세요.
- **체크리스트:**
 - ☐ `.github/workflows` 디렉토리를 만들었나요?
 - ☐ 워크플로우 파일이 main 브랜치에 푸시되었나요?
 - ☐ Actions 탭에서 워크플로우가 실행되었음을 확인했나요?
 - ☐ 빌드/테스트가 성공적으로 완료되었나요?

3교시: AWS ECS / GCP GKE로 배포 실습

개념 설명

컨테이너 오케스트레이션은 다수의 컨테이너를 대규모로 배포·관리하기 위해 네트워킹, 스케줄링, 확장 등을 자동화하는 기술입니다 ¹⁰. AWS와 GCP는 각각 컨테이너 오케스트레이션 서비스를 제공합니다. **AWS Elastic Container Service(ECS)**는 AWS가 자체 개발한 관리형 컨테이너 오케스트레이션 서비스로, 다수의 서버에 컨테이너를 배포하고 관리할 수 있게 합니다 ¹¹. ECS는 AWS 서비스와 높은 통합성을 갖추고 있어, AWS 콘솔 또는 API로 간편하게 클러스터, 작업정의(Task Definition), 서비스 등을 구성할 수 있습니다 ¹¹. 한편, **Google Kubernetes Engine(GKE)**는 오픈소스 Kubernetes를 구글이 관리형으로 제공하는 서비스로, Google 인프라 상에서 Kubernetes 클러스터를 쉽고 신뢰성 있게 운영할 수 있게 해줍니다 ¹².

예를 들어, AWS의 ECS에서는 컨테이너 설정을 **Task Definition**으로 정의한 뒤 서비스(Service)를 실행하면 해당 컨테이너가 클러스터에 배포됩니다 ¹³. 반면 GKE에서는 Kubernetes **Deployment** 리소스에 컨테이너 이미지를 지정하면, 다수의 파드(Pod)가 생성되어 애플리케이션이 실행됩니다 ¹⁴ ¹².

! [아키텍처 그림 삽입 위치]

(위: 컨테이너 오케스트레이션 예시 - AWS ECS와 GCP GKE의 개념 비교)

실습 목표

- AWS ECS와 GCP GKE 환경에서 동일한 컨테이너 애플리케이션을 배포합니다.
- 클러스터 생성, 컨테이너 레지스트리 등록, 애플리케이션 배포 과정을 익힙니다.

실습 절차

1. 컨테이너 이미지 준비:

2. 이전 실습에서 만든 Docker 이미지를 AWS ECR과 GCP GCR에 각각 푸시합니다.
 - **AWS ECR:** AWS 콘솔에서 ECR 리포지토리를 생성하고, `aws ecr get-login-password` 명령으로 Docker 로그인 후 이미지 푸시.
 - **GCP GCR:** `gcloud auth configure-docker` 로 Docker 로그인 후 `docker push gcr.io/[PROJECT-ID]/myapp:v1` 명령으로 이미지 푸시.
3. **AWS ECS 배포:**
4. **ECS 클러스터 생성:** AWS 콘솔 또는 CLI로 ECS 클러스터를 생성합니다. (EC2 모드 또는 Fargate 모드 선택)
5. **Task Definition 작성:** JSON 형식으로 Task Definition을 작성합니다. 예시(간략화):

```
{
  "family": "myapp-task",
  "containerDefinitions": [
    {
      "name": "myapp",
      "image": "<AWS_ACCOUNT_ID>.dkr.ecr.<region>.amazonaws.com/myapp:v1",
      "cpu": 256,
      "memory": 512,
      "essential": true,
      "portMappings": [{ "containerPort": 3000, "hostPort": 3000 }]
    }
  ]
}
```

6. **ECS 서비스 생성:** 클러스터 내에서 서비스(Service)를 생성하여 Task Definition으로부터 컨테이너를 실행합니다. 로드 밸런서(예: ALB)를 연결하여 외부 접근을 설정할 수 있습니다.
7. **서비스 실행 확인:** ECS 콘솔에서 서비스가 “Active” 상태가 되고, 연결된 로드밸런서의 도메인 혹은 퍼블릭 IP로 접속 시 애플리케이션이 정상 동작하는지 확인합니다.
8. **GCP GKE 배포:**
9. **GKE 클러스터 생성:** Google Cloud 콘솔 또는 `gcloud` CLI로 GKE 클러스터를 생성합니다. (예: `gcloud container clusters create my-cluster --zone us-central1-a`)
10. **이미지 푸시:** 이미 `gcr.io/[PROJECT-ID]/myapp:v1` 으로 이미지를 푸시한 상태여야 합니다.
11. **Kubernetes 매니페스트 작성:** Deployment와 Service YAML 파일을 작성합니다. 예) deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: gcr.io/PROJECT-ID/myapp:v1
```

```
ports:
- containerPort: 3000
```

12. 배포 및 서비스 생성:

```
kubectl apply -f deployment.yaml
kubectl expose deployment myapp-deployment --type=LoadBalancer --port=3000
```

13. 배포 확인: `kubectl get services` 명령으로 LoadBalancer 타입 서비스의 External IP를 확인하고, 해당 IP로 접속 시 애플리케이션에 접근됩니다.

14. 리소스 정리: (옵션) 실습 후 생성된 클러스터, 서비스, 리소스는 필요에 따라 삭제합니다.

실습 코드 예시

• AWS Task Definition 예시: (위 절차와 유사)

```
{
  "family": "myapp-task",
  "containerDefinitions": [
    {
      "name": "myapp",
      "image": "123456789012.dkr.ecr.us-west-1.amazonaws.com/myapp:v1",
      "cpu": 256,
      "memory": 512,
      "essential": true,
      "portMappings": [{ "containerPort": 3000, "hostPort": 3000 }]
    }
  ]
}
```

• GKE Deployment YAML 예시: (위 절차와 유사)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: gcr.io/my-gcp-project/myapp:v1
```



```
ports:
- containerPort: 3000
```

예상 결과

- **AWS ECS:** ECS 콘솔에서 새로 생성한 클러스터와 서비스가 “ACTIVE” 상태로 표시되고, 각 태스크(Task)가 실행 중임을 확인할 수 있습니다. 서비스에 연결한 ALB의 도메인 이름 또는 퍼블릭 IP를 웹 브라우저에서 열면 애플리케이션이 정상 실행됩니다.
- **GCP GKE:** `kubectl get pods` 명령으로 2개의 파드가 정상 기동되었음을 확인합니다.
`kubectl get svc` 명령으로 생성된 LoadBalancer 서비스의 External IP를 확인할 수 있으며, 이 IP로 접속하면 애플리케이션 화면이 나타납니다.

요약 박스

- **혼자 해보기:**
- ECS 서비스의 **스케일 조정**을 시도해 보세요. (예: 서비스 탭에서 Desired count를 늘려보거나 CLI로 `update-service` 사용)
- GKE에서 `kubectl scale deployment/myapp-deployment --replicas=3` 명령으로 복제 수를 변경해 보세요.
- **퀴즈:**
- AWS ECS에서 **Task Definition**은 무엇을 설명하나요?
- GKE에서 **ReplicaSet**과 **Deployment**의 관계는 무엇인가요?
- ECS와 EKS, GKE 중 Kubernetes 기반이 아닌 서비스는 무엇인가요?
- **체크리스트:**
- [] AWS ECR에 이미지를 정상 푸시했나요?
- [] ECS 클러스터와 서비스가 생성되었나요?
- [] GKE 클러스터가 생성되고 인증되었나요?
- [] Deployment와 Service로 애플리케이션이 실행되었나요?

4교시: 전체 자동 배포 파이프라인 구성 실습

개념 설명

전체 자동 배포 파이프라인(CI/CD 파이프라인)은 코드 커밋부터 빌드, 테스트, 컨테이너 이미지 배포, 실제 환경(클러스터) 배포까지 모든 단계를 자동화한 흐름입니다. 예를 들어 GitHub에 코드를 `push` 하면, GitHub Actions가 해당 커밋을 기반으로 이미지를 빌드하여 컨테이너 레지스트리에 업로드하고, 이어서 AWS ECS나 GCP GKE에 자동으로 배포하게 할 수 있습니다. 이러한 자동화된 파이프라인은 인적 개입을 최소화하여 빠른 배포와 높은 안정성을 보장합니다

6 10 .

실습에서는 GitHub Actions를 활용하여 앞에서 만든 Docker 이미지를 AWS ECS와 GCP GKE로 각각 자동 배포해 봅니다. GitHub Secrets에 AWS 자격증명, GCP 서비스 계정 키 등을 설정한 뒤, 워크플로우 파일(.github/workflows/deploy.yml)을 작성하여 배포를 완성합니다.

! [아키텍처 그림 삽입 위치]

(위: 자동 배포 파이프라인 예시 - GitHub Actions로 코드 푸시 → Docker 빌드/푸시 → AWS ECS/GCP GKE 배포)

실습 목표

- GitHub Actions로 코드가 푸시되면 자동으로 AWS ECS와 GCP GKE에 배포되는 파이프라인을 구성합니다.
- GitHub Secrets를 설정하고, 워크플로우 파일을 작성하여 CI/CD를 구현해 봅니다.

실습 절차

1. GitHub Secrets 설정:

2. AWS: IAM 사용자의 Access Key ID/Secret Key를 `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY` 라는 이름으로 저장합니다.

3. GCP: 서비스 계정 JSON 파일 내용을 `GCP_SA_KEY` 같은 이름으로 저장하고, `GCP_PROJECT_ID` 를 따로 Secret으로 저장합니다.

4. 워크플로우 파일 작성 (AWS용):

```
name: CI-CD to ECS
on:
  push:
    branches: [ main ]
jobs:
  deploy-ecs:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v2
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: us-west-1
      - name: Login to ECR
        uses: aws-actions/amazon-ecr-login@v1
      - name: Build and Push to ECR
        run: |
          docker build -t ${ steps.login-ecr.outputs.registry }/myapp:latest .
          docker push ${ steps.login-ecr.outputs.registry }/myapp:latest
      - name: Deploy to ECS
        uses: aws-actions/amazon-ecs-deploy-task-definition@v1
        with:
          task-definition: ecs-task-def.json
          service: myapp-service
          cluster: my-cluster
          wait-for-service-stability: true
```

위 예시는 AWS 자격증명 설정 후 Amazon ECR에 로그인하여 이미지를 빌드·푸시하고, `amazon-ecs-deploy-task-definition` 액션으로 ECS에 배포합니다 [15](#) [16](#) .

5. 워크플로우 파일 작성 (GCP용):

```
name: CI-CD to GKE
on:
  push:
    branches: [ main ]
jobs:
  deploy-gke:
    runs-on: ubuntu-latest
```

```

steps:
  - name: Checkout
    uses: actions/checkout@v4
  - name: Setup GCloud SDK
    uses: google-github-actions/setup-gcloud@v1
    with:
      project_id: ${{ secrets.GCP_PROJECT_ID }}
      service_account_key: ${{ secrets.GCP_SA_KEY }}
  - name: Configure Docker with GCloud
    run: gcloud auth configure-docker
  - name: Build and Push to GCR
    run: |
      docker build -t gcr.io/${{ secrets.GCP_PROJECT_ID }}/myapp:latest .
      docker push gcr.io/${{ secrets.GCP_PROJECT_ID }}/myapp:latest
  - name: Deploy to GKE
    uses: google-github-actions/get-gke-credentials@v1
    with:
      cluster_name: my-cluster
      location: us-central1-a
      credentials: ${{ secrets.GCP_SA_KEY }}
  - name: kubectl apply
    run: |
      kubectl set image deployment/myapp-deployment myapp=gcr.io/${{ secrets.GCP_PROJECT_ID }}/myapp:latest
      kubectl rollout status deployment/myapp-deployment

```

GCP 예시에서는 `setup-gcloud` 로 CLI를 설정하고, Docker 이미지를 GCR에 푸시한 뒤 `kubectl` 로 GKE 클러스터에 롤링 업데이트합니다 ¹⁷ ¹⁸ .

6. **커밋 후 배포 확인:** 워크플로우 파일을 저장소에 커밋하면 두 파이프라인이 실행됩니다. GitHub Actions 탭에서 ECS와 GKE 배포가 완료되었는지 확인합니다. AWS 콘솔과 GCP 콘솔에서 각각 새로운 이미지로 애플리케이션이 배포된 것을 확인합니다.
7. **검증:** 웹 브라우저에서 AWS ALB 도메인과 GKE 서비스의 External IP를 통해 애플리케이션이 정상 응답함을 확인합니다.

실습 코드 예시

• AWS용 GitHub Actions 예시: (핵심 스텝)

```

- name: Login to Amazon ECR
  uses: aws-actions/amazon-ecr-login@v1
- name: Build and push image
  run: |
    docker build -t $ECR_REGISTRY/myapp:$GITHUB_SHA .
    docker push $ECR_REGISTRY/myapp:$GITHUB_SHA
- name: Fill new image in task definition
  uses: aws-actions/amazon-ecs-render-task-definition@v1
  with:
    task-definition: ecs-task-def.json
    container-name: myapp
    image: ${{ steps.build-image.outputs.image }}
- name: Deploy to ECS

```

```

uses: aws-actions/amazon-ecs-deploy-task-definition@v1
with:
  task-definition: ${{ steps.task-def.outputs.task-definition }}
  service: myapp-service
  cluster: my-cluster
  wait-for-service-stability: true

```

(`amazon-ecs-render-task-definition` 과 `deploy-task-definition` 액션을 통해 이미지를 적용 및 배포합니다 16 19 .)

• GCP용 GitHub Actions 예시: (핵심 스텝)

```

- uses: google-github-actions/setup-gcloud@v1
with:
  project_id: ${{ secrets.GCP_PROJECT_ID }}
  service_account_key: ${{ secrets.GCP_SA_KEY }}
- run: gcloud auth configure-docker
- run: |
  docker build -t gcr.io/$PROJECT_ID/myapp:$GITHUB_SHA .
  docker push gcr.io/$PROJECT_ID/myapp:$GITHUB_SHA
- uses: google-github-actions/get-gke-credentials@v1
with:
  cluster_name: my-cluster
  location: us-central1-a
  credentials: ${{ secrets.GCP_SA_KEY }}
- run: |
  kubectl set image deployment/myapp-deployment myapp=gcr.io/$PROJECT_ID/myapp:
  $GITHUB_SHA
  kubectl rollout status deployment/myapp-deployment

```

(이미지 빌드/푸시 후 `kubectl` 로 GKE에 배포합니다 17 18 .)

예상 결과

- **ECS 배포:** GitHub Actions가 실행되며, AWS ECS 서비스가 새 이미지로 업데이트됩니다. AWS ECS 콘솔에서 서비스 이벤트에 “Deregistered old version, added new version”과 같은 로그가 확인되고, 애플리케이션이 계속 운영됩니다.
- **GKE 배포:** Actions가 실행되며 `kubectl apply` 를 통해 Deployment가 업데이트됩니다. GKE 콘솔이나 `kubectl rollout status` 명령으로 새로운 버전으로 파드가 교체되는 것을 확인할 수 있습니다.
- 최종적으로 AWS와 GCP 모두 최신 애플리케이션이 배포되어 웹 브라우저 접속 시 새로운 기능 또는 버전이 반영되어 나타납니다.

요약 박스

- **혼자 해보기:**
- GitHub Actions 워크플로우를 수정하여 **Pull Request** 기준으로만 배포되도록 혹은 특정 태그를 푸시할 때만 배포되도록 설정해 보세요.
- AWS의 경우 Fargate 모드로 변경하여 서버리스 방식 배포를, GCP의 경우 `kubectl rollout undo` 명령으로 이전 버전으로 되돌리기를 시험해 보세요.
- **퀴즈:**
- GitHub Actions에서 **Secret**을 사용하는 이유는 무엇인가요?

- `aws-actions/configure-aws-credentials` 액션은 어떤 역할을 하나요?
- GKE에 배포할 때 어떤 정보(환경변수)를 설정해야 하나요?
- **체크리스트:**
 - [] GitHub Secrets에 AWS/GCP 자격증명이 올바르게 등록되었나요?
 - [] 워크플로우 파일이 `.github/workflows` 에 저장되었나요?
 - [] 코드 푸시 후 GitHub Actions가 정상 실행되었나요?
 - [] AWS ECS 서비스와 GKE 파드가 최신 이미지로 업데이트되었나요?

출처: Docker 개념 및 비교 1 3 , Docker Compose 설명 5 , GitHub Actions 워크플로우 구조 7 8 , CI/CD 개념 6 , AWS ECS 개요 11 , GKE 개요 12 , 컨테이너 오케스트레이션 정의 10 , AWS/GCP 배포 실습 절차 20 21 , GitHub Actions를 통한 ECS/GKE 배포 예시 15 16 17 18 .

1 2 What is a Container? | Docker

<https://www.docker.com/resources/what-container/>

3 4 컨테이너 및 도커 개념정리

<https://velog.io/@geunwoobaek/%EC%BB%A8%ED%85%8C%EC%9D%B4%EB%84%88-%EB%B0%8F-%EB%8F%84%EC%BB%A4-%EA%B0%9C%EB%85%90%EC%A0%95%EB%A6%AC>

5 Docker Compose:우아한 다중 컨테이너 오케스트레이션

<https://velog.io/@gorachelgogo/Docker-Compose%EC%9A%B0%EC%95%84%ED%95%9C-%EB%8B%A4%EC%A4%91-%EC%BB%A8%ED%85%8C%EC%9D%B4%EB%84%88-%EC%98%A4%EC%BC%80%EC%8A%A4%ED%8A%B8%EB%A0%88%EC%9D%B4%EC%85%98>

6 What is CI/CD?

<https://www.redhat.com/en/topics/devops/what-is-ci-cd>

7 8 9 Building a CI/CD Workflow with GitHub Actions | GitHub Resources - GitHub Resources

<https://resources.github.com/learn/pathways/automation/essentials/building-a-workflow-with-github-actions/>

10 What is Container Orchestration? - Container Orchestration Explained - AWS

<https://aws.amazon.com/what-is/container-orchestration/>

11 13 AWS ECS vs EKS의 차이, DevOps 엔지니어가 알려드립니다. | 이랜서 블로그

<https://www.elancer.co.kr/blog/detail/840>

12 GKE 개요 | Google Kubernetes Engine (GKE) | Google Cloud

<https://cloud.google.com/kubernetes-engine/docs/concepts/kubernetes-engine-overview?hl=ko>

14 20 21 클라우드 플랫폼 (AWS, GCP) 배포 - 안동민 개발노트

<https://andongmin.com/docs/nest/ch11/ch11-3>

15 16 19 Deploying to Amazon Elastic Container Service - GitHub Docs

<https://docs.github.com/en/actions/how-tos/deploy/deploy-to-third-party-platforms/amazon-elastic-container-service>

17 18 Deploying to Google Kubernetes Engine - GitHub Docs

<https://docs.github.com/ko/actions/how-tos/managing-workflow-runs-and-deployments/deploying-to-third-party-platforms/deploying-to-google-kubernetes-engine>